# Estrutura de Dados

## Algoritmos de Busca e Ordenação

**Prof. Leonardo Albernaz Amaral**
IFRS - Restinga

2026/02



https://rebrand.ly/ed_262

# Conteúdo

- **Algoritmos de Busca**
  - **Busca Linear** (Desordenada)
    - **Algoritmo de "Ordenação"** (BubbleSort)
  - **Busca Linear Ordenada**
  - **Busca Binária** (Divisão e Conquista)
  - Exercícios práticos **+ Trabalho 6** (Análise de desempenho de buscas)
- **Algoritmos de Ordenação**
  - Algoritmos quadráticos: **BubbleSort**, **SelectionSort** e **InsertionSort**
  - Algoritmo Eficiente: **QuickSort** (recursividade)
  - Exercícios práticos **+ Trabalho 7** (Análise de desempenho de ordenações) **+ Trabalho 8** (Estudos de casos)

# Algoritmos de Busca

# Algoritmos de Busca
## (Definição e tipos de algoritmos)

- **Definição**
  - Busca é o processo de recuperação de dados armazenados em um repositório ou estrutura de dados.
- **Fatores que influenciam a escolha do algoritmo**
  - **Estrutura dos dados (o local da busca)**: Vetores (arrays), listas encadeadas, árvores, tabelas hash ou dados não estruturados.
  - **Ordenação prévia**: os dados estão ordenados logicamente ou desordenados?
  - **Duplicidade**: a estrutura contém valores duplicados ou não.
  - ... e também **tamanho** (o número de elementos) **da estrutura** a ser buscada.

# Algoritmos de Busca

## (Definição e tipos de algoritmos)

- **Métodos de busca:**
  - **Dados Desordenados:**
    - Busca Linear (Sequencial)
  - **Dados Ordenados:**
    - Busca Linear Ordenada (com parada antecipada)
    - Busca Binária

# Algoritmos de Busca
## (Busca Linear)

### Busca Linear (características):

- **Pré-requisito:**
  - Não exige que o vetor esteja ordenado (elementos desordenados ou em ordem arbitrária).
- **Funcionamento / Comportamento:**
  - Percorre o vetor inteiro de forma sequencial (do início ao fim), comparando o elemento pesquisado com cada elemento do vetor até encontrar o elemento desejado ou esgotar a busca.
- **Complexidade Assintótica ($O$):**
  - **Pior Caso:** Complexidade Linear - $O(n)$ (quando o elemento procurado está na última posição ou não existe no vetor, exigindo a varredura completa de todos os elementos).
  - **Melhor Caso:** Complexidade Constante - $O(1)$ (quando o elemento procurado está logo na primeira posição do vetor).
  - **Caso Médio:** Complexidade Linear - $O(n)$ (quando o elemento é encontrado estatisticamente na metade das posições).

# Algoritmos de Busca
## (Busca Linear)

### Busca Linear (características):

- **Vantagens:**
  - Extrema simplicidade de implementação e lógica direta.
  - Funciona perfeitamente em qualquer tipo de dado ou estrutura, **sem exigir nenhuma preparação ou ordenação prévia**.
- **Desvantagens:**
  - Altamente ineficiente e lento para grandes volumes de dados, pois o tempo de execução cresce linearmente com o tamanho da base.

# Algoritmos de Busca
## (Busca Linear)

- **Implementação da busca linear em Python:**

```python
def busca_linear(vetor: list[int], elemento: int) -> int:
    """
    Percorre o vetor sequencialmente comparando cada elemento.
    Retorna o índice do elemento se encontrado, ou -1 caso contrário.
    """
    for i in range(len(vetor)):
        if vetor[i] == elemento:
            return i
    return -1
```

**Site com demonstração visual do algoritmo:**
https://coddy.tech/visualize/pt/searching/linear-search

# Algoritmos de Busca
## (Busca Linear)

- **Prática da busca linear.**
  - Crie um programa em Python (busca_linear.py) que implemente o método “busca_linear” do slide anterior e também o código principal para testá-lo.
    - Para o teste, inicialize dinamicamente um vetor de **50 posições** com números inteiros aleatórios de **1 a 100**, garantindo que não tenha números duplicados.
      - Dica em Python: use random.sample(range(1, 101), 50).
    - Chame a função busca_linear(), trate o retorno e exiba o resultado de forma clara.
- **Especialização de Testes:**
  - Amplie o programa para testar vetores maiores: 1.000, 5.000, 10.000, 50.000, 100.000, 500.000 e 1.000.000 posições, modularizando o código.
  - Com base nos testes realizados com diferentes tamanhos de vetores, como você descreveria o comportamento deste algoritmo em termos de número de passos e escalabilidade ao lidar com grandes massas de dados?

# Algoritmos de Busca
## (Busca Ordenada)

### Busca Ordenada:

- **Pré-requisito:**
  - Exige obrigatoriamente que o vetor esteja previamente ordenado (elementos ordenados logicamente).
- **Funcionamento / Otimização:**
  - Similar ao “linear”, mas com “condição de parada antecipada”.
  - **Parada Antecipada:** Permite encerrar a busca antes de percorrer todo o vetor se o elemento atual for maior que o valor procurado, pois os elementos seguintes serão ainda maiores.
- **Complexidade Assintótica ($O$):**
  - **Pior Caso:** Complexidade Linear - $O(n)$ (quando o elemento procurado é o maior de todos ou não existe, ou é maior que o último elemento do vetor).
  - **Melhor Caso:** Complexidade Constante - $O(1)$ (quando o elemento está na primeira posição).
  - **Caso Médio:** Complexidade Lienar - $O(n)$ (apresenta melhor desempenho prático em buscas sem sucesso e no caso médio devido ao corte antecipado do laço).

# Algoritmos de Busca

## (Busca Ordenada)

### Busca Ordenada:

- **Vantagens:**
  - Melhora o desempenho médio em relação à busca linear comum (especialmente em buscas sem sucesso), graças à lógica de interrupção antecipada.
- **Desvantagens:**
  - Mantém a mesma complexidade assintótica de pior caso ($O(n)$ - Complexidade Linear).
  - Depende rigidamente de o vetor estar ordenado para funcionar corretamente.

# Algoritmos de Busca
## (Busca Ordenada)

- **Implementação da busca ordenada:**

```python
def busca_linear_ordenada(vetor: list[int], elemento: int) -> int:
    """
    Recebe um vetor ordenado e o elemento pesquisado.
    Retorna o índice se encontrado, ou -1 se não encontrar ou ultrapassar o valor.
    """
    for i in range(len(vetor)):
        if vetor[i] == elemento:
            return i
        elif vetor[i] > elemento:
            # Parada antecipada: elementos futuros serão ainda maiores
            break
    return -1
```

# Algoritmos de Busca
## (Busca Ordenada)

### Algoritmo de ordenação BubbleSort:

- **Pré-requisito:**
  - Aplicável a vetores desordenados.
- **Funcionamento / Otimização:**
  - Compara pares de elementos adjacentes e os troca de lugar se estiverem na ordem errada. O maior elemento "flutua" progressivamente para o final do vetor a cada passada.
  - Basicamente repete o processo de ordenação conforme o tamanho do vetor. Se tamanho igual a 10, poderá repetir até 10 vezes (no pior caso).
  - Conta com otimização de parada antecipada se nenhuma troca ocorrer em uma passada completa.
- **Complexidade Assintótica ($O$):**
  - **Pior Caso:** Complexidade Quadrádica - $O(n^2)$ (vetor em ordem inversa/decrescente, exigindo o número máximo de comparações e trocas).
    - Exige dois laços de repetição para percorrer e comparar os elementos do vetor.
  - **Melhor Caso:** Complexidade Linear - $O(n)$ (vetor já totalmente ordenado, detectado na primeira passada pela bandeira de controle).
  - **Caso Médio:** Complexidade Quadrática - $O(n^2)$ (vetor com elementos dispostos de forma puramente randômica).
    - Exige dois laços de repetição para percorrer e comparar os elementos do vetor.

# Algoritmos de Busca
## (Busca Ordenada)

### Algoritmo de ordenação BubbleSort:

- **Vantagens:**
  - Conceito muito simples de entender e implementar, sendo excelente para introduzir a lógica de ordenação e trocas (*swaps*).
  - Identifica rapidamente vetores já ordenados (no melhor caso).
- **Desvantagens:**
  - Desempenho catastrófico em grandes massas de dados devido ao crescimento quadrático ($O(n^2)$ - Complexidade Quadrática).

# Algoritmos de Busca
## (Busca Ordenada)

**Implementação do bubbleSort**

```python
def bubble_sort(vetor: list[int]) -> None:
    fim = len(vetor)
    continua = True

    # Enquanto houver trocas E o limite for maior que 1...
    while continua and fim > 1:
        continua = False  # Assume que está ordenado nesta passada
        # Laço para levar o maior elemento encontrado para final do vetor...
        for i in range(fim - 1): # o "i" determina a janela de interação/comparação
            if vetor[i] > vetor[i + 1]:
                # Troca os elementos de lugar (Swap)
                vetor[i], vetor[i + 1] = vetor[i + 1], vetor[i]
                continua = True  # Houve troca, logo o vetor não estava ordenado
        # Reduz o tamanho da varredura, pois o maior elemento já foi levado para o fim
        fim -= 1
```

**Site com demonstração visual do algoritmo:**

https://coddy.tech/visualize/sorting/bubble-sort?lang=pt&view=bars&speed=0.5&size=14

# Algoritmos de Busca
## (Busca Ordenada)

- **Prática da busca ordenada.**
  - Crie um programa em Python (busca_linear_ordenada.py) que implemente o método “busca_ordenada” dos slides anteriores e também o código principal para testá-lo.
    - Para o teste, inicialize dinamicamente um vetor de **50 posições** com números inteiros aleatórios de **1 a 100**, garantindo que não tenha números duplicados.
      - Dica em Python: use random.sample(range(1, 101), 50).
    - **Não esqueça de ordenar o vetor com o método “bubbleSort()”** antes de realizar a busca.
    - Chame a função busca_linear_ordenada(), trate o retorno e exiba o resultado de forma clara.
  - **Especialização de Testes:**
    - Amplie o programa para testar vetores maiores: 1.000, 5.000, 10.000, 50.000, 100.000, 500.000 e 1.000.000 de posições, modularizando o código.
    - Com base nos testes realizados com diferentes tamanhos de vetores, como você descreveria o comportamento deste algoritmo em termos de número de passos e escalabilidade ao lidar com grandes massas de dados?

# Algoritmos de Busca
## (Busca Binária)

### Busca Binária:

- **Pré-requisito:**
  - Exige obrigatoriamente que o vetor esteja ordenado.
- **Funcionamento / Otimização:**
  - Baseia-se no paradigma de **Divisão e Conquista**.
  - O algoritmo divide repetidamente o espaço de busca ao meio (calculando o índice central), descartando metade do vetor a cada comparação (metade esquerda ou metade direita).
  - O método compara o elemento procurado com o elemento central.
- **Complexidade Assintótica ($O$):**
  - **Pior Caso:** Complexidade Logarítmica - $O(\log n)$ (quando o elemento está nas extremidades extremas ou não existe após todas as subdivisões possíveis).
    - Significa que o número de passos necessários para resolver o problema cresce de forma **extremamente lenta** em comparação com o crescimento do volume de dados ($n$), visto que a cada iteração (passo), o algoritmo **divide o espaço de busca exatamente ao meio**, descartando 50% dos dados restantes.
  - **Melhor Caso:** Complexidade Constante - $O(1)$(quando o elemento está exatamente na posição central exata do vetor logo na primeira verificação).
  - **Caso Médio:** Complexidade Logarítmica - $O(\log n)$ (extremamente eficiente e escalável para grandes volumes de dados).

# Algoritmos de Busca
## (Busca Binária)

**Implementação da busca binária**

```python
def busca_binaria(vetor: list[int], elemento: int) -> int:
    """
    Realiza a busca binária iterativa em um vetor ordenado.
    Retorna o índice do elemento ou -1 se não encontrado.
    """
    inicio = 0
    fim = len(vetor) - 1

    while inicio <= fim:
        meio = (inicio + fim) // 2

        if vetor[meio] == elemento:
            return meio
        elif elemento < vetor[meio]:
            fim = meio - 1  # Descarta metade direita
        else:
            inicio = meio + 1  # Descarta metade esquerda

    return -1
```

**Site com demonstração visual do algoritmo:**
https://coddy.tech/visualize/searching/binary-search?lang=pt&view=array&speed=0.5&size=14

# Algoritmos de Busca
## (Busca Binária)

- **Prática da busca binária.**
  - Crie um programa em Python (busca_binaria.py) que implemente o método “busca_binaria” do slide anterior e também o código principal para testá-lo.
    - Para o teste, inicialize dinamicamente um vetor de **50 posições** com números inteiros aleatórios de **1 a 100**, garantindo que não tenha números duplicados
      - Dica em Python: use random.sample(range(1, 101), 50).
    - Não esqueça de ordenar o vetor com o método “bubbleSort()” antes de realizar a busca.
    - Chame a função busca_binaria(), trate o retorno e exiba o resultado de forma clara.
  - **Especialização de Testes:**
    - Amplie o programa para testar vetores maiores: 1.000, 5.000, 10.000, 50.000, 100.000, 500.000 e 1.000.000 de posições, modularizando o código.
    - Com base nos testes realizados com diferentes tamanhos de vetores, como você descreveria o comportamento deste algoritmo em termos de número de passos e escalabilidade ao lidar com grandes massas de dados?

# Algoritmos de Busca

## (Trabalho sobre algoritmos de busca)

### Trabalho 6

- **Objetivo:**
  - Medir e comparar o desempenho prático de execução dos 3 algoritmos de busca estudados (“Busca Linear”, “Busca Linear Ordenada” e “Busca Binária”) utilizando o módulo time (time.perf_counter()).
- **Cargas de Teste:**
  - Tamanhos de vetor: 1.000, 5.000, 10.000, 50.000, 100.000, 500.000 e 1.000.000 de elementos.
- **Diretrizes Metodológicas:**
  - **As medições de tempo devem isolar estritamente a chamada da função de busca** (desprezando a geração de números e a ordenação prévia).
  - Reutilizar exatamente o mesmo conjunto de dados gerado para todos os algoritmos na mesma rodada de testes.
- **Entregas (para dia 23/10):**
  - Código-fonte documentado, instruções claras de execução e relatório analítico de desempenho.

# Algoritmos de Ordenação

# Algoritmos de Ordenação
## (Algoritmos Básicos)

**Algoritmos Básicos – Complexidade Quadrática ($O(n^2)$):**

- **Bubble Sort (já foi visto)**
- **Selection Sort (Ordenação por Seleção):**
- **Insertion Sort (Ordenação por Inserção):**

- Todos possuem complexidade quadrática $O(n^2)$ no pior caso, sendo inadequados para grandes volumes de dados.

# Algoritmos de Ordenação

## (Algoritmos Básicos)

### SelectionSort (Ordenação por Seleção)

- **Pré-requisito:**
  - Aplicável a vetores desordenados.
- **Funcionamento:**
  - Divide o vetor logicamente em duas sub-listas: ordenada (à esquerda) e não ordenada (à direita). Encontra repetidamente o menor elemento da parte não ordenada e o troca com o primeiro elemento daquela parte. Ao final, o inclui no início da lista ordenada
- **Complexidade Assintótica ($O$):**
  - **Pior Caso:** Complexidade Quadrática - $O(n^2)$.
  - **Melhor Caso:** Complexidade Quadrática - $O(n^2)$(realiza exatamente a mesma quantidade de varreduras e comparações independentemente da ordenação inicial do vetor).
  - **Caso Médio:** Complexidade Quadrática - $O(n^2)$.

# Algoritmos de Ordenação
## (Algoritmos Básicos)

### SelectionSort (Ordenação por Seleção)

- **Vantagens:**
  - Realiza o menor número absoluto de trocas (*swaps*) em comparação com outros algoritmos quadráticos, pois faz no máximo uma troca por passada completa.
- **Desvantagens:**
  - O número de comparações é sempre fixo e alto, mesmo que o vetor já esteja totalmente ordenado (não aproveita o melhor caso).

# Algoritmos de Ordenação

## (Algoritmos Básicos)

## Selection Sort:

```python
def selection_sort(vetor: list[int]) -> None:

    n = len(vetor)
    # Passa por cada posição do vetor assumindo que ela receberá o menor elemento restante
    for i in range(n):
        # Assume que o elemento atual (i) é o menor da sub-lista não ordenada
        min_index = i
        # Explora o restante do vetor à direita para encontrar um elemento ainda menor
        for j in range(i + 1, n):
            if vetor[j] < vetor[min_index]:
                # Atualiza o índice do menor elemento encontrado
                min_index = j
        # Se o menor elemento não for o próprio elemento da posição i, faz a troca
        if min_index != i:
            vetor[i], vetor[min_index] = vetor[min_index], vetor[i]
```

**Site com demonstração visual do algoritmo:**

https://coddy.tech/visualize/pt/sorting/selection-sort

Compara o primeiro elemento com os demais até achar o menor, e quando acha, joga o menor para o início da lista.

Com isso, ele divide o vetor em duas partes: a **sub-lista ordenada** (à esquerda) e a **sub-lista não ordenada** (à direita).

A cada passo, encontra o menor elemento da parte não ordenada e o troca com o primeiro elemento daquela parte.

# Algoritmos de Ordenação
## (Algoritmos Básicos)

### InsertionSort (Ordenação por Inserção)

- **Pré-requisito:**
  - Aplicável a vetores desordenados.
- **Funcionamento:**
  - Constrói a lista ordenada item por item, retirando um elemento e inserindo-o na posição correta dentro da sub-lista já ordenada à esquerda (semelhante ao manuseio de cartas de baralho).
- **Complexidade Assintótica ($O$):**
  - **Pior Caso:** Complexidade Quadrática - $O(n^2)$(vetor em ordem estritamente inversa, exigindo deslocamentos máximos para cada elemento).
  - **Melhor Caso:** Complexidade Linear - $O(n)$ (vetor já ordenado; cada elemento é comparado apenas uma vez com o anterior e permanece na posição, sem deslocamentos).
  - **Caso Médio:** Complexidade Linear - $O(n^2)$.

# Algoritmos de Ordenação

## (Algoritmos Básicos)

### InsertionSort (Ordenação por Inserção)

- **Vantagens:**
  - Altamente eficiente e prático para conjuntos de dados pequenos ou para listas que já estão **parcialmente ordenadas** (caso em que atinge o tempo linear $O(n)$).
- **Desvantagens:**
  - Ineficiente para grandes volumes de dados totalmente desordenados devido ao custo dos deslocamentos.

# Algoritmos de Ordenação

## (Algoritmos Básicos)

## Insertion Sort:

```python
def insertion_sort(vetor: list[int]) -> None:
    # Começamos a iterar a partir do segundo elemento (índice 1).
    for i in range(1, len(vetor)):
        # Guarda o valor do elemento atual que será inserido na parte ordenada
        chave = vetor[i]
        # Inicializa o ponteiro 'j' para o último elemento da sub-lista ordenada à esquerda
        j = i - 1
        # Desloca os elementos da sub-lista ordenada que são maiores que a chave
        # uma posição para a direita, abrindo espaço para a inserção
        while j >= 0 and vetor[j] > chave:
            vetor[j + 1] = vetor[j]  # Deslocamento para a direita
            j -= 1  # Move o ponteiro para o elemento anterior

        # Insere a chave na sua posição correta encontrada na sub-lista ordenada
        vetor[j + 1] = chave
```

Constrói a lista ordenada elemento por vez, retirando um elemento da entrada e inserindo-o na posição correta dentro da sub-lista já ordenada (semelhante ao modo como organizamos cartas de baralho na mão).

**Site com demonstração visual do algoritmo:**

https://coddy.tech/visualize/pt/sorting/insertion-sort

# Algoritmos de Ordenação
## (Algoritmo Eficiente)

### Algoritmo de Ordenação Eficiente

- **QuickSort (Divisão e Conquista + Recursividade)**
- **Complexidade:** $O(n \log n)$ - Linear-Logarítmica no caso médio/melhor, e $O(n^2)$ - Quadrática no pior caso (pivô mal escolhido).

# Algoritmos de Ordenação
## (Algoritmo Eficiente)

### QuickSort

- **Pré-requisito:**
  - Aplicável a vetores desordenados.
- **Funcionamento / Otimização:**
  - Utiliza o paradigma de **Divisão e Conquista** e recursividade. Escolhe um elemento como pivô, particiona o vetor em sub-conjuntos de elementos menores e maiores que o pivô, e aplica o processo recursivamente.
- **Complexidade Assintótica ($O$):**
  - **Pior Caso:** Complexidade Quadrática - $O(n^2)$ (quando a escolha do pivô é sistematicamente ruim — como pegar sempre o menor ou maior elemento —, gerando partições extremamente desbalanceadas).
  - **Melhor Caso:** Complexidade Linearítmica (ou Linear-Logarítmica) $O(n \log n)$ (quando o pivô divide o vetor perfeitamente ao meio em todas as etapas da recursão).
  - **Caso Médio:** Complexidade Linearítmica (ou Linear Logarítmica) $O(n \log n)$ (quando a escolha do pivô divide o vetor em partes **razoavelmente equilibradas**, ou seja, nem perfeitamente ao meio como no melhor caso, nem de forma totalmente desbalanceada como no pior caso).

# Algoritmos de Ordenação

## (Algoritmo Eficiente)

### QuickSort

- **Vantagens:**
  - Extraordinariamente rápido e eficiente no caso médio para grandes volumes de dados, sendo um dos algoritmos base mais utilizados na indústria.
- **Desvantagens:**
  - O desempenho degrada severamente para $O(n^2)$ se a estratégia de escolha do pivô for mal dimensionada para dados específicos.
  - Consome pilha de memória devido às chamadas recursivas (guardar o estado das variáveis de controle de cada recursão + endereços de retorno) – o que pode ter crescimento logarítmico ($O(\log n)$) ou linear ($O(n)$).

# Algoritmos de Ordenação

## (Algoritmo Eficiente)

## QuickSort (O(n log n)) – com list comprehensions:

```python
def quick_sort(vetor: list[int]) -> list[int]:
    # Caso base da recursão: vetores com 0 ou 1 elemento já estão ordenados
    if len(vetor) <= 1:
        return vetor

    # Escolha do pivô: pegamos o elemento central do vetor atual
    pivo = vetor[len(vetor) // 2]
    # Particionamento utilizando List Comprehensions do Python:
    # 1. Cria uma lista apenas com os elementos menores que o pivô
    menores = [x for x in vetor if x < pivo]
    # 2. Cria uma lista com os elementos iguais ao pivô (trata duplicatas corretamente)
    iguais = [x for x in vetor if x == pivo]
    # 3. Cria uma lista apenas com os elementos maiores que o pivô
    maiores = [x for x in vetor if x > pivo]

    # Chamada recursiva combinando as partes menores ordenadas, os iguais,
    # e as partes maiores ordenadas
    return quick_sort(menores) + iguais + quick_sort(maiores)
```

**O que faz:** Varre cada elemento x dentro do vetor e guarda na nova lista menores apenas aqueles que são **estritamente menores** que o pivo.(*"Colete cada elemento x do vetor se x for menor que o pivô"*)

**Filtra e guarda todos os elementos que possuem o mesmo valor do pivô.**

Varre o vetor e mantem apenas os elementos que são **estritamente maiores** que o pivô.

Versão com list comprehensions.

Usa List Comprehension (compreensão de lista) para fazer a etapa de particionamento do QuickSort de forma funcional e extremamente concisa.

**Site com demonstração visual do algoritmo:**
https://coddy.tech/visualize/sorting/quick-sort?lang=pt&view=bars&speed=0.5&size=14

# Algoritmos de Ordenação

## (Algoritmo Eficiente)

### QuickSort (O(n log n)) – clássico (unificado):

```python
def quick_sort(vetor: list[int], baixo: int, alto: int) -> None:
    """
    Implementação unificada e clássica do QuickSort in-place (Lomuto).
    Ordena o vetor diretamente na memória, sem criar listas auxiliares.
    """
    if baixo < alto:
        # 1. Particionamento embutido
        pivo = vetor[alto]
        i = baixo - 1

        for j in range(baixo, alto):
            if vetor[j] <= pivo:
                i += 1
                vetor[i], vetor[j] = vetor[j], vetor[i]

        # Coloca o pivô na sua posição definitiva
        vetor[i + 1], vetor[alto] = vetor[alto], vetor[i + 1]
        indice_pivo = i + 1

        # 2. Chamadas recursivas para as duas metades
        quick_sort(vetor, baixo, indice_pivo - 1)
        quick_sort(vetor, indice_pivo + 1, alto)
```

Como essa versão usa índices (baixo e alto), para ordenar um vetor inteiro, digite:
**quick_sort(meu_vetor, 0, len(meu_vetor) - 1)**

**Abordagem *In-Place* (No Local):**

- Ao contrário da versão funcional, esta implementação reorganiza os elementos **diretamente dentro do mesmo vetor** na memória, eliminando completamente a criação de listas auxiliares e otimizando o uso de recursos.

**O Papel dos Índices (baixo e alto):**

- Em vez de fatiar fisicamente o vetor, o algoritmo utiliza parâmetros de controle de índices (baixo para o "início" e alto para o "fim") para isolar e processar apenas a sub-fatia ativa da estrutura.

**A Mecânica do Particionamento:**

- **Escolha do Pivô:** Seleciona o elemento final do trecho atual (vetor[alto]).
- **Varredura por Ponteiros (i e j):** O ponteiro j percorre o vetor comparando os valores. Sempre que encontra um elemento menor ou igual ao pivô, ele incrementa a fronteira i e efetua uma troca direta (*swap*).
- **Posicionamento Definitivo:** No final do laço, o pivô é encaixado na sua posição exata entre os menores e os maiores.

**Recursividade (Divisão e Conquista):**

- O algoritmo divide o problema e chama a si mesmo para repetir o processo de forma independente nas duas metades geradas (esquerda e direita do pivô), até que os trechos fiquem reduzidos a 0 ou 1 elemento.

# Algoritmos de Ordenação
## (Algoritmo Eficiente)

**Comparação das 2 versões...**

- Versão funcional (com *list comprehensions*)
- Versão clássica unificada

- **Semelhanças**
  - Ambas as versões partilham exatamente a mesma **complexidade de tempo** teórica:
    - **Caso Médio e Melhor Caso:** $O(n \log n)$(Complexidade Linearítmica)
    - **Pior Caso:** $O(n^2)$(Complexidade Quadrática)
- **Diferenças**
  - Elas diferem no **espaço (consumo de memória auxiliar)**:
    - **As versões clássicas:** Consomem pouquíssima memória extra, precisando apenas de $O(\log n)$de espaço de pilha para gerenciar as chamadas recursivas. (Não criam novas listas – gerenciam ponteiros)
    - **A versão funcional:** Teoricamente ainda é $O(n \log n)$ em tempo, mas na prática **ela é muito mais lenta e consome muita memória**, pois precisa alocar novos objetos de lista no heap do computador a cada passo da recursão.

# Algoritmos de Ordenação

## (Trabalho sobre algoritmos de ordenação)

### Trabalho 7

- **Objetivo:**
  - Desenvolver um projeto em Python para testar e comparar o desempenho computacional de **4 algoritmos de ordenação** (BubbleSort, SelectionSort, InsertionSort e QuickSort).
- **Cargas de Teste:**
  - Tamanhos de vetor: 1.000, 10.000, 50.000, 100.000, 500.000 e 1.000.000 de elementos gerados aleatoriamente.
- **Diretrizes Metodológicas:**
  - Medições estritamente focadas no tempo de execução do método de ordenação (desprezando a alocação inicial de memória).
  - Utilizar exatamente o mesmo conjunto de dados de teste para garantir justiça na comparação algorítmica.
- **Entregas (para dia 23/10):**
  - Código modularizado, instruções de execução e relatório técnico comparativo com gráficos ou tabelas de tempo e suas justificativas de desempenho e comparações entre os diferentes métodos.

# Algoritmos de Ordenação

## (Trabalho sobre algoritmos de ordenação)

### Trabalho 8

- **Objetivo:**
  - Analisar **cenários reais**, escolher estrategicamente e implementar **algoritmos de busca e de ordenação** aplicados ao domínio escolhido pelo grupo.
  - Justificar **técnica e computacionalmente** a escolha do algoritmo com base no desempenho e no comportamento esperado da massa de dados.
- **Entregáveis (via GoogleDrive) (para dia 23/10):**
  - Arquivo principal em Python (.py) com os algoritmos de busca e ordenação devidamente implementados na respectiva área de código.
  - Relatório técnico resumido com a justificação das escolhas estratégicas feitas para o domínio.
  - Apresentação e demonstração prática do funcionamento do sistema em aula (para dia 23/10).

# Algoritmos de Ordenação

## (Trabalho sobre algoritmos de ordenação)

### Domínios a serem escolhidos (Trabalho 8)

- **1. Jogo Urbano (Check-ins):** Jogador (id_jogador, nome, pontuacao).
  - **Desafio:** Analisar as consultas por ID numérico e por nome parcial. Decidir e implementar as melhores estratégias de busca e ordenação do ranking de pontuação para grandes massas de dados.
- **2. E-commerce de Livros: Livro (codigo_barras, titulo, preco, avaliacoes).**
  - **Desafio:** Avaliar o comportamento dos dados para decidir entre busca linear ou binária na consulta por código de barras e título parcial, escolhendo o algoritmo de ordenação ideal para alternar entre preço e avaliações.
- **3. Secretaria Acadêmica: AlunoMatricula (matricula, nome, nota_final, idade).**
  - **Desafio:** Definir a estratégia de busca para consultas de matrícula e escolher o algoritmo de ordenação adequado para reordenar a lista de espera ponderando a nota final e demais critérios do edital.
- **4. Plataforma de Streaming (VOD): Filme (id_filme, titulo, genero, visualizacoes).**
  - **Desafio:** Estabelecer a melhor abordagem algorítmica para buscas por ID exato versus termos textuais flexíveis, bem como a ordenação eficiente do catálogo completo com base no número de visualizações.
- **5. Gestão de Aeroporto (Painel): Voo (numero_voo, destino, horario_partida, portao).**
  - **Desafio:** Escolher a estratégia de busca ideal para localizar voos por número e definir como o painel deve manter a ordenação cronológica das partidas de forma otimizada.
- **6. Triagem Hospitalar: Paciente (senha_atendimento, nome, nivel_urgencia, tempo_espera_min).**
  - **Desafio:** Determinar o método de busca adequado para verificar senhas de atendimento e selecionar o algoritmo de ordenação que melhor gerencie a fila de prioridades do pronto-socorro.

# Algoritmos de Ordenação

## (Trabalho sobre algoritmos de ordenação)

Detalhamento dos Problemas **(Trabalho 8)**

- **Proposta 1: Jogo Urbano (Check-ins)**
- **Contexto do Problema:** O sistema gerencia os jogadores de um aplicativo de exploração urbana. Ele precisa processar milhares de check-ins em pontos turísticos, atualizar pontuações e gerar diferentes relatórios em tempo real.
- **O Desafio para o Grupo:**
  - **Estratégia de Busca:** O sistema precisa buscar o perfil de um jogador pelo ID numérico versus buscar jogadores pelo nome parcial digitado na barra de pesquisa. *Qual algoritmo de busca você aplicará para cada caso e por quê?*
  - **Estratégia de Ordenação:** O painel exibe o ranking geral dos jogadores por pontuação, além de destacar periodicamente os melhores da semana. *Como você gerenciará a ordenação dessa massa de dados para manter o sistema responsivo?*

# Algoritmos de Ordenação

## (Trabalho sobre algoritmos de ordenação)

Detalhamento dos Problemas **(Trabalho 8)**

- **Proposta 2: E-commerce de Livros**
- **Contexto do Problema:** Uma plataforma digital precisa gerenciar um catálogo massivo de produtos (livros). Os usuários entram para buscar itens e filtrá-los de acordo com preferências de preço e avaliação.
- **O Desafio para o Grupo:**
  - **Estratégia de Busca:** Os clientes realizam buscas diretas por código de barras (ISBN) e buscas aproximadas por título ou autor. *Que tipo de algoritmo de busca se encaixa melhor em cada uma dessas frentes de consulta?*
  - **Estratégia de Ordenação:** O usuário pode alternar a visualização do catálogo por "Menor Preço", "Mais Vendidos" e "Lançamentos". *Qual estratégia de ordenação você adotará para suportar essas visualizações com alta eficiência?*

# Algoritmos de Ordenação

## (Trabalho sobre algoritmos de ordenação)

Detalhamento dos Problemas **(Trabalho 8)**

- **Proposta 3: Secretaria Acadêmica**
- **Contexto do Problema:** Uma universidade precisa processar a lista de espera de candidatos para disciplinas concorridas. A lista possui critérios rígidos de ordenação (nota no exame, idade, ordem de inscrição).
- **O Desafio para o Grupo:**
  - **Estratégia de Busca:** A secretaria precisa verificar rapidamente se um aluno específico já está na lista usando o número de matrícula. *Como a estrutura da lista influenciará a escolha entre uma busca linear ou binária?*
  - **Estratégia de Ordenação:** Periodicamente, a lista precisa ser reordenada combinando múltiplos critérios. *Qual algoritmo de ordenação deve ser escolhido para garantir que a reclassificação de milhares de candidatos ocorra sem gargalos de desempenho?*

# Algoritmos de Ordenação

## (Trabalho sobre algoritmos de ordenação)

Detalhamento dos Problemas **(Trabalho 8)**

- **Proposta 4: Plataforma de Streaming (VOD)**
- **Contexto do Problema:** Um serviço de streaming precisa gerenciar o acervo digital de filmes e séries, controlando o metadado de visualizações, categorias e recomendações personalizadas.
- **O Desafio para o Grupo:**
  - **Estratégia de Busca:** O sistema precisa de acesso direto a um título específico por ID de mídia, mas também precisa suportar filtros textuais flexíveis por gênero ou sinopse. *Justifique quais algoritmos de busca atenderão a esses dois cenários distintos.*
  - **Estratégia de Ordenação:** O catálogo precisa ser reorganizado dinamicamente por "Mais Assistidos" e "Lançamentos". *Como você avalia o custo computacional de reordenar catálogos massivos e qual algoritmo você selecionou para resolver o problema?*

# Algoritmos de Ordenação

## (Trabalho sobre algoritmos de ordenação)

Detalhamento dos Problemas **(Trabalho 8)**

- **Proposta 5: Gestão de Aeroporto**
- **Contexto do Problema:** Um terminal aeroportuário de grande fluxo precisa gerenciar em tempo real o painel eletrônico de partidas e chegadas de aeronaves.
- **O Desafio para o Grupo:**
  - **Estratégia de Busca:** A equipe de solo precisa localizar rapidamente um voo específico informando o código numérico do voo versus localizar voos por destino textualmente. *Quais algoritmos de busca você implementará para garantir respostas ágeis ao operador?*
  - **Estratégia de Ordenação:** O painel precisa manter os voos rigorosamente ordenados por ordem cronológica de partida. *Você deve reordenar a estrutura completa a cada alteração ou existe uma abordagem mais inteligente com base nos algoritmos estudados?*

# Algoritmos de Ordenação

## (Trabalho sobre algoritmos de ordenação)

Detalhamento dos Problemas **(Trabalho 8)**

- **Proposta 6: Triagem Hospitalar**
- **Contexto do Problema:** Um hospital gerencia a fila de espera de pacientes no pronto-socorro baseando-se em senhas de atendimento e na gravidade clínica (classificação de risco).
- **O Desafio para o Grupo:**
  - **Estratégia de Busca:** A recepção precisa verificar instantaneamente o status de um paciente na base utilizando o número da senha de atendimento. *Qual algoritmo de busca é ideal e quais pré-requisitos estruturais ele exige da sua lista?*
  - **Estratégia de Ordenação:** A lista de espera precisa ser reordenada continuamente ponderando critérios de urgência médica combinados com a ordem de chegada. *Qual algoritmo de ordenação garante a melhor performance e integridade para este cenário crítico?*

# FIM